

GBI CLI & Python

A practical guide to moving, archiving and restoring research data.

COPY keeps your originals.
MOVE verifies before removing eligible filesystem sources.
RESTORE brings an archive back to files and folders.
Object Storage originals are always retained.

Start with a small copy. Check the paths and result. Then use the same tools for your finished datasets and checkpoints.

User guide for GBI 0.4.43
05 November 2026

Find what you need

Start here 3

Copy, move and durable storage 4

Archives without tuning flags 4

Selecting files 5

Execution and preview 5

Prefect route 6

Load data into the cache before a job 7

Status and recovery 8

Python SDK 9

Cached usage 10

Expert options 10

Troubleshooting and records 12

Examples use placeholder paths. Run `gbi data roots` and substitute your own paths before copying a command. The installed `--help` describes the version on your cluster.

GBI copies, moves, archives and restores your research data between Lustre, FSS and Object Storage. It runs with your Unix permissions and verifies the destination before removing a source. Moving Object Storage data to Lustre or FSS asks before removing it.

This guide describes the **0.4.43 release**, including retry by transfer ID for ordinary file and link transfers. Archive moves reuse freshly computed source hashes while file identity and metadata remain unchanged. They retain 0.4.17's single destination archive check and parallel final chunk verification. Full destination verification and verified receipts still precede ordinary source removal. Journal-only resumes hash each remaining source file once per invocation, then reuse that verified observation before unlink; persisted journal digests alone are never reused, and a hardlink ctime change forces a rehash. The 0.4.11 transfer behaviour was accepted for ordinary users on 28 September 2026. Check `gbi --version` and the README before using new options. Its matching typed broker and production FSS/Lustre archive and restore flows are deployed and accepted, including automatic `--archive --prefect`.

Start here

Load the module and find your storage paths:

```
module load gbi
gbi --version
gbi data roots
```

`roots` prints your Lustre scratch, FSS home and Object Storage roots with a line on what belongs in each, the team folders you are a member of and the read-only collections you can open. The Object Storage root is your member folder in your team's bucket; accounts from before team folders also reach it through their `/mnt/user-data` path, and `roots` names that path when you have it. Replace the placeholder paths below with paths printed by `roots`.

```
# Copy files and keep the originals.
gbi data copy /your/lustre/results /your/object/results

# Move finished files, removing verified Lustre sources.
gbi data move /your/lustre/results /your/object/results

# Archive a finished directory; GBI chooses packaging and part sizes.
gbi data move /your/lustre/experiment /your/object/experiment --archive

# Restore the complete archive directory to ordinary files.
gbi data copy /your/object/experiment /your/lustre/restored-experiment
```

For a directory, its contents go inside the destination directory you name. For one file, the destination is its new filename, unless that destination already is a directory; then GBI keeps the original filename inside it.

An existing identical destination can be verified and reused. A different existing file is retained and reported as a collision, never overwritten silently. Stop all writers before moving a finished dataset.

Normal help shows everyday choices. Use `gbi data copy --help` or `gbi data move --help`. Expert overrides for existing scripts are described under `--help-all` and at the end of this guide.

Copy, move and durable storage

`copy` retains the source. `move` removes each selected source only after independent destination readback, a verification receipt and source identity checks. A packed move also verifies the stored archive before removing the selected original entries.

Source → destination	`move`
Lustre or FSS → anywhere	removes the source after verification
Object Storage → Object Storage	removes the source after verification
Object Storage → Lustre or FSS	asks `[y/N]` on a terminal; `--delete-source` removes it without asking; with no terminal and no flag it copies, keeps the source and prints one line saying so
a source you cannot delete	refused before anything is copied

`copy --delete-source` is refused. A mount itself, such as `/mnt/team/my-lab`, is never a removable source; move a folder inside it. A partial archive restore keeps its container, and unselected archive members are never removed.

Every selected regular file is hashed while reading the source and again while reading the destination. Symlinks are represented without following their targets. Unsupported special files fail safely. A worker exit status, file size or destination that merely exists is not deletion evidence.

A large transfer is not an atomic whole-directory transaction. Some units can be verified and complete while others are still running or have failed. Use the terminal summary and receipts to determine whether the entire dataset is complete; do not treat a partly populated destination as a finished backup.

Archives without tuning flags

Use `--archive` when you want a directory packaged for storage. Leave it off when you want directly readable loose files. No packing threshold or chunk-size choice is needed.

GBI keeps the destination you name as the single layout root. It packs child directories into uncompressed GBI tar archives. If one exceeds the archive metadata limit, GBI splits it further while preserving its original relative paths. Root-level files, links and empty directories remain represented at their original locations. Large packed units use verified resumable parts automatically. Chunking the archive preserves its stored member metadata.

For example, a source containing `samples/` and `manifest.tsv` can be stored as `samples.gbi.tar` and `manifest.tsv` under the chosen destination. Restoring that destination recreates `samples/` and `manifest.tsv`, not an extra competing directory tree. A chunked archive is recognized automatically as well. Do not rename generated containers inside a larger archive directory.

The ordinary route records the complete archive partition on Lustre before starting payload writes. A retry reuses that partition even when earlier units have already removed their verified sources. It does not repartition the remaining files into a conflicting layout. A nonempty destination without a matching saved plan is rejected for inspection.

```
gbi data copy /your/lustre/experiment /your/object/experiment --archive
gbi data copy /your/object/experiment /your/fss/restored-experiment
```

Restoring needs no archive or chunking flags. You can also restore one generated archive into an explicitly named directory:

```
gbi data copy /your/object/experiment/samples.gbi.tar /your/lustre/samples
```

GBI archives preserve member relative paths, empty directories, safe symlinks, ordinary modes, modification times and hardlink relationships within a container. They do not preserve ownership, ACLs, extended attributes, privileged mode bits or hardlinks across different containers. A hardlink whose other name is excluded becomes a regular file. Existing destination directory permissions are preserved.

Timestamp verification permits truncation of less than one second. Lustre restores have shown whole-second timestamps; do not depend on exact nanosecond preservation. The archive's outer source directory is the layout root, not a stored member with its own restored metadata.

A marked GBI archive is recognized even if explicitly named with a different suffix. An ordinary unmarked tar file is copied as a file, not extracted. Within a directory, retain generated `.gbi.tar`, `.gbi.tar.gz` and `.gbi-chunks` names so discovery can recognize the containers.

Selecting files

Use repeatable `--include` and `--exclude` filters. They match filenames recursively, are case-sensitive, and use shell-style globs. Includes form a union; an exclusion always wins. Quote globs so your shell passes them unchanged.

```
gbi data move /your/lustre/run /your/object/run --archive \
  --include '*.pt' --include '*.pt.*' --exclude '*.tmp'
```

This selects `model.pt` and `model.pt.ready.json` at any depth, but not `model.ptx`. Unselected files stay in the source. Without includes, files are selected by default unless excluded.

The same filters can select members during restore:

```
gbi data copy /your/object/run /your/lustre/selected-run --include '*.pt'
```

For an archive, filters match member filenames recursively. For a chunked regular file, they match the original filename recorded in its manifest. Object Storage retains the full original even for a partial restore.

Execution and preview

GBI normally chooses the execution location for you:

- Small plain transfers can run in your shell after a bounded metadata probe. The usual foreground limit is 8 GiB, configurable by the site.

- Larger or slow-to-discover work goes to Slurm. Automatic archive planning runs on an execution node, not as a recursive login-node scan.
- Inside an existing Slurm allocation, ordinary CLI and Python calls reuse that allocation.
- `--detach` explicitly submits another Slurm job. Add `--wait` if your script needs to wait for it.
- `--prefect` sends the transfer to the site's Prefect service instead of running it in your shell or a Slurm job of your own.

```
gbi data copy SOURCE DESTINATION --dry-run
gbi data move SOURCE DESTINATION --archive --detach --wait
```

An ordinary `--dry-run` writes no files and submits no job. For automatic archives it explains the policy and destination root; the complete saved partition is constructed on the execution node when you start the transfer. It does not claim a huge directory has already been fully inventoried.

A foreground Ctrl-C stops foreground work. Ctrl-C while following a submitted Slurm job only detaches the display; the job continues. Reconnect using the printed transfer ID. Automatic thresholds are not completion-time promises.

Prefect route

Use `--prefect` for large transfers through the site's identity-bound broker. No Prefect UI or cloud credentials are needed in your script. It accepts the same paths as an ordinary transfer: any path you can read to any path you can write on Lustre, FSS and the Object Storage mounts (your member folder, your team's shared folder, read-only collections as a source), with Object Storage on at least one side. Between two Object Storage mounts the copy happens inside Object Storage. GBI sends the resolved paths; the transfer job checks, as you, that the whole source is readable and the destination writable before any data moves, and names the first path that is not.

```
gbi data copy /your/fss/experiment \
    /mnt/team/my-lab/members/$USER/saved-experiment --prefect --wait
gbi data copy /mnt/team/my-lab/members/$USER/saved-experiment \
    /mnt/team/my-lab/shared/experiment --prefect --wait
gbi data move /mnt/team/my-lab/members/$USER/saved-experiment \
    /your/lustre/restored --prefect --delete-source
```

The default writes individual objects. The deployed typed broker and production flows accept renamed FSS destinations, exclusions, restore reservation overrides, portable archive options and automatic `--archive --prefect`. The CLI refuses unsupported requested options before submission; do not remove a filter just to bypass that refusal.

Plain `--prefect --dry-run` is a local preview and submits nothing. Supported packing/chunking previews submit a managed metadata-only plan: they create tracking records, but no payload writes, receipts or source deletions. Add `--wait` or inspect the printed run ID.

Inside a Slurm allocation, Prefect still submits a separate managed job. Allow for its queue time in the calling job's time limit. It does not reuse your allocation.

For archive destination reuse, Prefect requires its expected integrity metadata. Objects uploaded through Alluxio or another tool may lack it. Use the ordinary route to verify and reuse those destinations. Prefect

restores still verify ordinary objects without that archive-upload metadata.

Load data into the cache before a job

Object Storage mounts serve a file's blocks from the storage workers' cache once they are resident, so a job that reads warm blocks runs at cache speed instead of pulling cold data from Object Storage. `gbi data cache` makes a path resident ahead of the job that needs it:

```
# First step of your job script: return when the blocks are cached.
gbi data cache /mnt/team/my-lab/members/$USER/dataset

# A shared reference, with a wait limit (the load itself continues).
gbi data cache /mnt/team/my-lab/shared/reference --timeout 900

# Sequencing data: a run folder, and only the files you need.
gbi data cache /mnt/instrument-data/illumina_nextseq_1/RUN --include '*.fastq.gz,*.bam'

# Submit and come back later.
gbi data cache /mnt/instrument-data/oxford_nanopore_promethion_1 --detach
gbi data status TRANSFER_ID --watch

# Inspect current cache state for a whole path, without submitting a load.
gbi data status /mnt/team/my-lab/shared/reference
```

Nothing is read on your node. The command resolves the path, the site's broker maps it to the storage cluster behind the mount, and the always-on cache service submits a native load without starting a Prefect flow. Every storage worker loads blocks in parallel, so the data lands in the cache faster than any single reader could pull it. The load applies the same permissions as your mount, from the top of the mount down: you can load what you can read, and nothing else. A folder is loaded whole when every file in it is yours to read and is in Object Storage; otherwise, and with `--include`, the readable matching files are loaded one by one. The path you name may be a symbolic link (the `/mnt/instrument-data/<instrument>` entries are links to the sequencing mounts); it is followed once and must land inside an Object Storage mount. Lustre and FSS paths are refused because they have no block cache. Running it again is cheap: resident blocks are not loaded again.

The command prints the cache state as it changes and, when the load ends, one line: files, bytes, the percentage resident, and any files skipped because you cannot read them or because they are still being written. Each path gets a transfer ID when your Lustre state directory is available; `gbi data status TRANSFER_ID --watch` reconnects with the original selection after `--detach`, a dropped session or `--timeout`. Ctrl-C only detaches; the load continues on the storage workers.

`gbi data status PATH` inspects current state for the whole path. Saved cache IDs from older releases still show their original Prefect flow state. Run warming under your own account in the job script; a root-run Slurm prolog cannot submit it on your behalf.

Exit codes: `0` the blocks are cached; `1` the load failed (the line says why, for example `refused: ... is not readable by ...`); `2` the request was refused before anything was submitted; `3` `--timeout` stopped the wait while the load continued. `--json` prints one summary line per command instead of progress. The cache is shared and evicts the least recently used blocks on its own, so a load is never refused for space; a very large dataset may tier older blocks down.

Status and recovery

```
gbi data status TRANSFER_ID
gbi data status TRANSFER_ID --watch
```

Use the printed transfer ID or a numeric Slurm ID when it identifies exactly one transfer. If an allocation contains multiple transfers, GBI lists their IDs rather than guessing. Exact transfer IDs remain usable after completed history publication. Existing jobs retain their saved code even after a new module is installed.

For a failed ordinary file or link transfer, keep its recovery state and retry using the transfer ID or an unambiguous Slurm job ID:

```
gbi data retry TRANSFER_ID
gbi data retry TRANSFER_ID --wait
```

Retry requires terminal history and refuses live or unresolved prior workers. It creates a fresh run linked to the original, using the saved paths, filters and deletion consent; the original history remains unchanged. A detached transfer stays detached; `--wait` follows it. An allocation transfer reuses your current allocation or submits a new Slurm job if you are outside one.

Existing files pass fresh checksum, destination reuse and owned-partial recovery checks. Across nodes, retry ignores only the mount's device ID and still requires the saved inode, type, size and modification time. Changed or replaced sources are retained. If the filesystem gives a file a different inode, inspect the history and use the original command for a fresh selection.

An absent source needs an acknowledged deletion receipt and a fresh destination checksum under the transfer lock. A verification receipt alone does not prove source removal. Missing or changed destinations fail safely. Output and new history distinguish copied, reused, already moved and failed files; already moved files count as reused and free no additional bytes. An absent empty directory must have its existing destination directory and receives no file or freed-space credit. A failed worker may have completed receipt-backed work before its result was lost; a failure count does not prove sources remain.

A saved finite selection is preserved. Older streaming requests have no complete saved file inventory: retry checks recorded files and rediscovers the original root and filters, including files not previously started. It reports that the original undiscovered remainder is unknown. An absent streaming source root needs completed discovery and matching receipt coverage.

Archive, packed and chunked transfers keep their original-command recovery path; ordinary retry refuses to reinterpret those manifests. Confirm the job and workers have stopped, inspect receipts and retained sources, then rerun the same command with the same source, destination and selection. A saved automatic archive plan preserves completed units and the original layout. Interrupted packed cleanup uses its saved original manifest, rechecks the destination and removes only unchanged selected entries still present; it can resume from a different node. Do not delete locks, journals, partial output or verification receipts to force a retry.

For a managed transfer, `gbi data retry` resends the identical saved Prefect request after a lost submission reply. It does not restart a failed flow. Inspect the existing run before submitting any new request.

A timeout observing the cluster does not prove a transfer stopped. Recheck the same job. `scancel JOB_ID` cancels an identified Slurm job; it does not undo earlier verified transfers or authorize deletion of incomplete sources.

Python SDK

Load the module before starting Python, including inside a virtual environment. The SDK calls the same installed CLI and waits for completion:

```
from pathlib import Path
from gbi import data

finished = Path("/your/lustre/finished-run")
stored = Path("/your/object/finished-run")

data.move(finished, stored, archive=True, include=["*.pt", "*.pt.*"])
data.copy(stored, "/your/lustre/restored-run")
```

Call after all writers have closed, normally once from rank zero. Use `data.copy` to retain filesystem originals. Both functions accept strings or `pathlib.Path`, stream progress to the current log and return `subprocess.CompletedProcess`. Failure raises `subprocess.CalledProcessError`; a missing executable raises `FileNotFoundError`. The SDK never silently retries a failed command.

A batch job can load the module and call that code directly:

```
#!/usr/bin/env bash
#SBATCH --job-name=train-and-archive
#SBATCH --cpus-per-task=2
#SBATCH --time=02:00:00

set -euo pipefail
module load gbi
python train_and_archive.py
```

Use `prefect=True` only when you want a separate managed transfer. Ordinary calls reuse the allocation.

Complete signatures, including compatible expert arguments:

```
copy(source, destination, *, include=(), exclude=(), pack=None,
     pack_small=False, chunk_size=None, dry_run=False, prefect=False,
     job_size=None, archive=False, delete_source=False)

move(source, destination, *, include=(), exclude=(), pack=None,
     pack_small=False, chunk_size=None, dry_run=False, prefect=False,
     delete_source=False, job_size=None, archive=False)
```

Filters accept one string or an iterable of strings. Boolean arguments must be actual booleans. `move(..., delete_source=True)` removes an Object Storage source moved to Lustre or FSS; the default keeps it, and the SDK never asks. `copy(..., delete_source=True)` is refused. Otherwise source removal is identical to the CLI.

Cached usage

`usage` starts with an aligned storage summary, then lists your largest Lustre folders. It never scans a filesystem itself:

```
gbi data usage
gbi data usage results --depth 2 --limit 20
```

`results` in the second example is a directory below your Lustre root, not a subcommand. Depth and limit must be positive.

```
Usage for /your/lustre

Storage summary
Tier        Used  Limit   Files  Measured by
------  --------  -----  ------  ----------------------------------
Lustre  85.2 GiB     --   2,888  live UID quota
FSS     12.0 GiB     --      --  OCI FSS quota accounting, 2d old

Lustre folder breakdown (complete snapshot 2026-09-27T02:00:00+00:00, 2d old)
Apparent total: 64.0 GiB across 2,876 inventory entries
Folder   Apparent size  Entries
-------  -------------  -------
results       60.0 GiB    2,000
code           4.0 GiB      876
```

Where the numbers come from:

- **Lustre** is your live UID quota from the Lustre client (`lfs`), for the whole filesystem. It is read every time you run the command. It counts the space allocated to your UID anywhere on Lustre, so it can be larger than the folder breakdown when you own files outside your root or Lustre has not yet freed deleted data, and smaller when your files are sparse (a file that reports 8 GiB but occupies no blocks is common in test fixtures).
- **FSS** is OCI's per-UID accounting for the shared FSS file system: logical data bytes you own, excluding snapshots, with no file count. The site records it when it publishes your snapshot, so its age is shown. It reads `not published` until your first snapshot includes it. If the weekly lookup fails, the row keeps last week's value with its real age and adds `(latest lookup failed)`; the next weekly run tries again.
- **The folder breakdown** comes from the site's weekly inventory of your Lustre root (Sunday 02:00 UTC). Sizes are apparent bytes, including directory and symlink entries, not allocated space. A snapshot marked `partial` could not open some folders in your root (for example ones owned by a service account); the warning says how many.

These are three different measurements, so they are not added together or compared as one total. A `Limit` of `--` means no limit applies.

Expert options

Normal archive use does not need these choices. Existing scripts may keep using them; `gbi data copy --help-all` exposes them.

Option	Ordinary / Slurm	Existing allocation	Prefect
`--archive`	Automatic directory archive	Yes	Production archive flows
`--include / --exclude`	Recursive filename globs	Yes	Up to 100 plain globs
`--pack tar/gzip`	One exact archive target	Yes	Archive only
`--pack-small`	Legacy selective small-file packing	Yes	Archive only
`--chunk-size SIZE`	Regular file or new archive	Yes	Archive only
`--job-size small/large`	No	No	Restore reservation override
`--local`	Requires allocation	Guard only	No
`--detach`	Submit another Slurm job	Submit another job	No
`--wait`	Wait for completion	Yes	Yes
`--dry-run`	No writes/submission	Yes	Local preview or supported managed plan
`--delete-source`	`move` only: removes an Object Storage source moved to Lustre or FSS without asking	Same	Same

Do not combine `--archive` with `--pack`, `--pack-small` or `--chunk-size`. Restore without encoding flags. `--detach`, `--local` and `--prefect` are mutually exclusive; so are `--pack` and `--pack-small`.

Exact archives and legacy small-file packing

`--pack tar` creates one uncompressed archive at the exact `.gbi.tar` target; `--pack gzip` uses an exact `.gbi.tar.gz` target. Explicit gzip is available when compression is worth its CPU cost. Automatic archives use tar; they do not guess that checkpoints compress well.

`--pack-small` is the older selective-packing mode. Its usual site defaults are at most 64 KiB per file, at least 40 files per group and at most 1 GiB per archive. Larger siblings remain loose. Its bounded `--dry-run` can report an incomplete lower-bound inventory; that is not authorization to archive or delete anything. Prefer `--archive` for a saved whole-directory layout.

The archive format limits both manifest and source-observation metadata to 64 MiB. Explicit whole-directory packing may exceed that limit even for a small payload. Automatic archives split such directories internally. A larger chunk size does not increase the metadata limit.

Explicit chunks and restore reservation

`--chunk-size 64MiB` writes verified resumable parts with a final manifest and completion marker. For a direct file, give the original destination name without `.gbi-chunks`; GBI adds that suffix. With `--prefect`, one regular file needs no size: a plain or `--archive` request for a file is uploaded as that same resumable store with the automatic part size, under the destination you name (a destination ending in `/` keeps the file's own name inside it). A single file takes no `--include` or `--exclude`. Whole-pack targets keep their `.gbi.tar` or `.gbi.tar.gz` suffix and gain `.gbi-chunks` as well. An incomplete store is not restorable before its completion marker validates.

Raw-file chunks preserve content and filename, not POSIX modes or timestamps. Archive chunks retain archive member metadata. Mutable journals and optional archive staging use configured Lustre scratch, not local temporary disk.

`--job-size small` requests 2 CPUs/16 GiB for a Prefect restore; `large` requests 8 CPUs/48 GiB. Omit it to retain the deployment setting. It changes neither the time limit nor the caller's allocation and does not guarantee immediate scheduling. Python uses `job_size="small"` or `job_size="large"`.

Troubleshooting and records

If a destination differs, keep it and inspect the collision. If a source changed, stop writers and inspect the saved request before retrying. If a site rejects a requested option, keep its intended selection and use a supported route rather than silently changing which files will move.

For a mount wait, worker deadline or missing terminal summary, retain the printed phase, path, receipt location and Slurm log. Confirm process state; do not infer completion or cancellation from a transport timeout.

Active state is private under `.gbi` on Lustre. Completed ordinary history is published under the user's Object Storage mount at `.gbi/transfers/PARENT_ID/TRANSFER_ID/`. The parent is the Slurm job/allocation ID, or the transfer ID for a foreground run. Follow the printed history path. Requests record code identity and route parameters; receipts record verification and deletion evidence. Prefect publishes its managed receipts through its own maintained flow.

An archive may finish before the rest of a dataset. A complete recovery means every selected path is accounted for, the destination is independently verified and its original directory layout is recoverable.